

Seg Beats

区间最值操作 & 区间历史最值

本文讲解吉老师在 [2016 年国家集训队论文](#) 中提到的线段树处理历史区间最值的问题。

区间最值

笼统地说，区间最值操作指，将区间 $[l, r]$ 的数全部对 x 取 $\max$ 或 $\min$ ，即 range_max 或者 range_min 。

▼ HDU5306 Gorgeous Sequence

维护一个序列 a ，执行以下操作：

1. $0\ l\ r\ t$ 。将 $a[l, r]$ 中的数全部对 t 取 $\max$ 。
2. $1\ l\ r$ 输出 $a[l, r]$ 中的最大值。
3. $2\ l\ r$ 输出 $a[l, r]$ 中的次大值。

多组测试数据，保证 $n, m \leq 10^6$ 。

区间取 $\max$ ，意味着只对那些大于 t 的数有更改。因此这个操作的对象不再是整个区间，而是「这个区间中大于 t 的数」。于是我们可以有这样的思路：每个结点维护该区间的最大值、次大值、区间和以及最大值的个数。接下来我们考虑区间对 t 取 $\max$ 的操作。

1. 如果 $t \geq \text{mx}$ ，显然这个操作是没有意义的，直接返回；
2. 如果 $t < \text{mx}$ ，那么这个操作就能更新当前区间中的最大值。于是我们让区间和加上 $(\text{mx} - t) \times \text{cnt}$ ，然后更新 mx 为 t ，并打一个标记。
3. 如果 $t < \text{se}$ ，那么这时你发现你不知道有多少个数涉及到更新的问题。于是我们的策略就是，暴力递归向下操作。然后上传信息。

这个算法的复杂度如何？使用势能分析法可以得到复杂度是 $O(n \log n)$ 的。具体分析过程见论文。

```
1 #include <algorithm>
2 #include <cstdio>
3 #include <iostream>
4 using namespace std;
5 constexpr int N = 1e6 + 6;
6
7 int t, n, m;
8 int a[N];
9 int mx[N << 2], se[N << 2], cn[N << 2], tag[N << 2];
10 long long sum[N << 2];
11
12 void pushup(int u) { // 向上更新标记
13     const int ls = u << 1, rs = u << 1 | 1;
14     sum[u] = sum[ls] + sum[rs];
15     if (mx[ls] == mx[rs]) {
16         mx[u] = mx[rs];
17         se[u] = max(se[ls], se[rs]);
18         cn[u] = cn[ls] + cn[rs];
19     } else if (mx[ls] > mx[rs]) {
20         mx[u] = mx[ls];
21         se[u] = max(se[ls], mx[rs]);
22         cn[u] = cn[ls];
23     } else {
24         mx[u] = mx[rs];
25         se[u] = max(mx[ls], se[rs]);
26     }
27 }
```

```

26     cn[u] = cn[rs];
27 }
28 }
29
30 void pushtag(int u, int tg) { // 单纯地打标记, 不暴搜
31     if (mx[u] <= tg) return;
32     sum[u] += (1ll * tg - mx[u]) * cn[u];
33     mx[u] = tag[u] = tg;
34 }
35
36 void pushdown(int u) { // 下传标记
37     if (tag[u] == -1) return;
38     pushtag(u << 1, tag[u]), pushtag(u << 1 | 1, tag[u]);
39     tag[u] = -1;
40 }
41
42 void build(int u = 1, int l = 1, int r = n) { // 建树
43     tag[u] = -1;
44     if (l == r) {
45         sum[u] = mx[u] = a[l], se[u] = -1, cn[u] = 1;
46         return;
47     }
48     int mid = (l + r) >> 1;
49     build(u << 1, l, mid), build(u << 1 | 1, mid + 1, r);
50     pushup(u);
51 }
52
53 void modify_min(int L, int R, int v, int u = 1, int l = 1, int r = n) {
54     if (mx[u] <= v) return;
55     if (L <= l && r <= R && se[u] < v) return pushtag(u, v);
56     int mid = (l + r) >> 1;
57     pushdown(u);
58     if (L <= mid) modify_min(L, R, v, u << 1, l, mid);
59     if (mid < R) modify_min(L, R, v, u << 1 | 1, mid + 1, r);
60     pushup(u);
61 }
62
63 int query_max(int L, int R, int u = 1, int l = 1, int r = n) { // 查询最值
64     if (L <= l && r <= R) return mx[u];
65     int mid = (l + r) >> 1, r1 = -1, r2 = -1;
66     pushdown(u);
67     if (L <= mid) r1 = query_max(L, R, u << 1, l, mid);
68     if (mid < R) r2 = query_max(L, R, u << 1 | 1, mid + 1, r);
69     return max(r1, r2);
70 }
71
72 long long query_sum(int L, int R, int u = 1, int l = 1, int r = n) { // 数值
73     if (L <= l && r <= R) return sum[u];
74     int mid = (l + r) >> 1;
75     long long res = 0;
76     pushdown(u);
77     if (L <= mid) res += query_sum(L, R, u << 1, l, mid);
78     if (mid < R) res += query_sum(L, R, u << 1 | 1, mid + 1, r);
79     return res;
80 }
81
82 void go() { // 根据题意
83     cin >> n >> m;
84     for (int i = 1; i <= n; i++) cin >> a[i];
85     build();
86     for (int i = 1; i <= m; i++) {
87         int op, x, y, z;
88         cin >> op >> x >> y;
89         if (op == 0)
90             cin >> z, modify_min(x, y, z);

```

```

91     else if (op == 1)
92         cout << query_max(x, y) << '\n';
93     else
94         cout << query_sum(x, y) << '\n';
95 }
96 }
97
98 signed main() {
99     cin.tie(nullptr)->sync_with_stdio(false);
100     cin >> t;
101     while (t--) go();
102     return 0;
103 }

```

▼ [BZOJ4695 最假女选手](#)

维护一个序列，执行以下操作：

1. 1 1 r x 。
2. 2 1 r x 。
3. 3 1 r x 。
4. 4 1 r 输出 。
5. 5 1 r 输出 。
6. 6 1 r 输出 。

。所有类型 操作有，其余操作满足。

同样的方法，我们维护最大、次大、最大个数、最小、次小、最小个数、区间和。除了这些信息，我们还需要维护区间、区间、区间加的标记。相比上一道题，这就涉及到标记下传的顺序问题了。我们采用这样的策略：

1. 我们认为区间加的标记是最优先的，其余两种标记地位平等。
2. 对一个结点加上一个 标记，除了用 更新卫星信息和当前结点的区间加标记外，我们用这个 v 更新区间 和区间的标记。
3. 对一个结点取 的（这里忽略暴搜的过程，假定标记满足添加的条件），除了更新卫星信息，我们要与区间 的标记做比较。如果 小于区间的标记，则所有的数最后都会变成 v，那么把区间的标记也变成 。否则不管。
4. 区间取 v 的 同理。

在维护信息的时候，当只有一个数或两个数的时候可能发生数集重合，比如一个数既是最大值又是次小值，需要特判。

```

1 #include <LTIostream>
2 using namespace std;
3
4 constexpr int N = 5e5 + 5, SZ = N << 2, INF = 0x7fffffff;
5
6 int n, m;
7 int a[N];
8
9 struct data {
10     int mx, mx2, mn, mn2, cmx, cmn, tmx, tmn, tad;
11     long long sum;
12 } t[SZ];
13
14 void pushup(int u) {
15     const int lu = u << 1, ru = u << 1 | 1;
16     t[u].sum = t[lu].sum + t[ru].sum;
17     if (t[lu].mx == t[ru].mx) {
18         t[u].mx = t[lu].mx, t[u].cmx = t[lu].cmx + t[ru].cmx;
19         t[u].mx2 = max(t[lu].mx2, t[ru].mx2);
20     } else if (t[lu].mx > t[ru].mx) {
21         t[u].mx = t[lu].mx, t[u].cmx = t[lu].cmx;
22         t[u].mx2 = max(t[lu].mx2, t[ru].mx);

```

```

23 } else {
24     t[u].mx = t[ru].mx, t[u].cmx = t[ru].cmx;
25     t[u].mx2 = max(t[lu].mx, t[ru].mx2);
26 }
27 if (t[lu].mn == t[ru].mn) {
28     t[u].mn = t[lu].mn, t[u].cmn = t[lu].cmn + t[ru].cmn;
29     t[u].mn2 = min(t[lu].mn2, t[ru].mn2);
30 } else if (t[lu].mn < t[ru].mn) {
31     t[u].mn = t[lu].mn, t[u].cmn = t[lu].cmn;
32     t[u].mn2 = min(t[lu].mn2, t[ru].mn);
33 } else {
34     t[u].mn = t[ru].mn, t[u].cmn = t[ru].cmn;
35     t[u].mn2 = min(t[lu].mn, t[ru].mn2);
36 }
37 }
38
39 void push_add(int u, int l, int r, int v) {
40     // 更新加法标记的同时,更新 $\min$ 和 $\max$ 标记
41     t[u].sum += (r - l + 1ll) * v;
42     t[u].mx += v, t[u].mn += v;
43     if (t[u].mx2 != -INF) t[u].mx2 += v;
44     if (t[u].mn2 != INF) t[u].mn2 += v;
45     if (t[u].tmx != -INF) t[u].tmx += v;
46     if (t[u].tmn != INF) t[u].tmn += v;
47     t[u].tad += v;
48 }
49
50 void push_min(int u, int tg) {
51     // 注意比较 $\max$ 标记
52     if (t[u].mx <= tg) return;
53     t[u].sum += (tg * 1ll - t[u].mx) * t[u].cmx;
54     if (t[u].mn2 == t[u].mx) t[u].mn2 = tg; // !!!
55     if (t[u].mn == t[u].mx) t[u].mn = tg; // !!!!!
56     if (t[u].tmx > tg) t[u].tmx = tg; // 更新取 $\max$ 标记
57     t[u].mx = tg, t[u].tmn = tg;
58 }
59
60 void push_max(int u, int tg) {
61     if (t[u].mn > tg) return;
62     t[u].sum += (tg * 1ll - t[u].mn) * t[u].cmn;
63     if (t[u].mx2 == t[u].mn) t[u].mx2 = tg;
64     if (t[u].mx == t[u].mn) t[u].mx = tg;
65     if (t[u].tmn < tg) t[u].tmn = tg;
66     t[u].mn = tg, t[u].tmx = tg;
67 }
68
69 void pushdown(int u, int l, int r) {
70     const int lu = u << 1, ru = u << 1 | 1, mid = (l + r) >> 1;
71     if (t[u].tad)
72         push_add(lu, l, mid, t[u].tad), push_add(ru, mid + 1, r, t[u].tad);
73     if (t[u].tmx != -INF) push_max(lu, t[u].tmx), push_max(ru, t[u].tmx);
74     if (t[u].tmn != INF) push_min(lu, t[u].tmn), push_min(ru, t[u].tmn);
75     t[u].tad = 0, t[u].tmx = -INF, t[u].tmn = INF;
76 }
77
78 void build(int u = 1, int l = 1, int r = n) {
79     t[u].tmn = INF, t[u].tmx = -INF; // 取极限
80     if (l == r) {
81         t[u].sum = t[u].mx = t[u].mn = a[l];
82         t[u].mx2 = -INF, t[u].mn2 = INF;
83         t[u].cmx = t[u].cmn = 1;
84         return;
85     }
86     int mid = (l + r) >> 1;
87     build(u << 1, l, mid), build(u << 1 | 1, mid + 1, r);

```

```

88  pushup(u);
89 }
90
91 void add(int L, int R, int v, int u = 1, int l = 1, int r = n) {
92     if (R < l || r < L) return;
93     if (L <= l && r <= R) return push_add(u, l, r, v); // !!! 忘 return
94     int mid = (l + r) >> 1;
95     pushdown(u, l, r);
96     add(L, R, v, u << 1, l, mid), add(L, R, v, u << 1 | 1, mid + 1, r);
97     pushup(u);
98 }
99
100 void tomin(int L, int R, int v, int u = 1, int l = 1, int r = n) {
101     if (R < l || r < L || t[u].mx <= v) return;
102     if (L <= l && r <= R && t[u].mx2 < v) return push_min(u, v); // BUG: 忘了返回
103     int mid = (l + r) >> 1;
104     pushdown(u, l, r);
105     tomin(L, R, v, u << 1, l, mid), tomin(L, R, v, u << 1 | 1, mid + 1, r);
106     pushup(u);
107 }
108
109 void tomax(int L, int R, int v, int u = 1, int l = 1, int r = n) {
110     if (R < l || r < L || t[u].mn >= v) return;
111     if (L <= l && r <= R && t[u].mn2 > v) return push_max(u, v);
112     int mid = (l + r) >> 1;
113     pushdown(u, l, r);
114     tomax(L, R, v, u << 1, l, mid), tomax(L, R, v, u << 1 | 1, mid + 1, r);
115     pushup(u);
116 }
117
118 long long qsum(int L, int R, int u = 1, int l = 1, int r = n) {
119     if (R < l || r < L) return 0;
120     if (L <= l && r <= R) return t[u].sum;
121     int mid = (l + r) >> 1;
122     pushdown(u, l, r);
123     return qsum(L, R, u << 1, l, mid) + qsum(L, R, u << 1 | 1, mid + 1, r);
124 }
125
126 long long qmax(int L, int R, int u = 1, int l = 1, int r = n) {
127     if (R < l || r < L) return -INF;
128     if (L <= l && r <= R) return t[u].mx;
129     int mid = (l + r) >> 1;
130     pushdown(u, l, r);
131     return max(qmax(L, R, u << 1, l, mid), qmax(L, R, u << 1 | 1, mid + 1, r));
132 }
133
134 long long qmin(int L, int R, int u = 1, int l = 1, int r = n) {
135     if (R < l || r < L) return INF;
136     if (L <= l && r <= R) return t[u].mn;
137     int mid = (l + r) >> 1;
138     pushdown(u, l, r);
139     return min(qmin(L, R, u << 1, l, mid), qmin(L, R, u << 1 | 1, mid + 1, r));
140 }
141
142 int main() {
143     cin.tie(nullptr)->sync_with_stdio(false);
144     cin >> n;
145     for (int i = 1; i <= n; i++) cin >> a[i];
146     build();
147     cin >> m;
148     for (int i = 1; i <= m; i++) {
149         int op, l, r, x;
150         cin >> op >> l >> r;
151         if (op <= 3) cin >> x;
152         if (op == 1)

```

```

153     add(l, r, x);
154     else if (op == 2)
155         tomax(l, r, x);
156     else if (op == 3)
157         tomin(l, r, x);
158     else if (op == 4)
159         cout << qsum(l, r) << '\n';
160     else if (op == 5)
161         cout << qmax(l, r) << '\n';
162     else
163         cout << qmin(l, r) << '\n';
164 }
165 return 0;
166 }

```

吉老师证出来这个算法的复杂度是 $O(n \log n)$ 的。

▼ Mzl loves segment tree

两个序列，一开始 a 中的数都是 0 。维护的操作是：

1. 对 a 做区间取
2. 对 a 做区间取
3. 对 a 做区间加
4. 询问 a 的区间和

每次操作完后，如果 a 的值发生变化，就给 a 加 1 。。

先考虑最容易的区间加操作。只要 a 那么整个区间的数都变化，所以给 B 作一次区间加即可。

对于区间取最值的操作，你发现你打标记与下传标记是与 a 数组一一对应的。本质上你将序列的数分成三类：最大值、最小值、非最值。并分别维护（只不过你没有建出具体的最值集合而已，但这并不妨碍维护的操作）。因此在打标记的时候顺便给 a 更新信息即可（注意不是给 a 打标记！是更新信息！）。查询的时候，你在 a 上查询，下传标记的时候顺便给 a 更新信息。找到需要的结点后，返回 a 的信息即可。这种操作本质上就是把最值的信息拿给 a 去维护了。另外仍要处理数集的重复问题。

▼ [CTSN loves segment tree](#)

维护两个序列，执行以下操作：

1. $1\ 1\ r\ x$ 。
2. $2\ 1\ r\ x$ 。
3. $3\ 1\ r\ x$ 。
4. $4\ 1\ r\ x$ 。
5. $5\ 1\ r$ 输出。

。

我们把区间 a 中的备选答案 分成四类：均不是序列 a 区间最大值、是序列 a 区间最大值但是 a 不是序列 a 区间最大值、不是序列 a 区间最大值但是 a 是序列 a 区间最大值、均是序列 a 区间最大值。我们不妨分别设为 l, r, s, t 。此外我们正常维护序列 a 的区间最大值和次大值。下传区间加法标记和 a 标记时对 a 最大值和次大值的处理与上述两个例题一致。对 a 的 a 标记会影响到 l 和 r ，对 a 的标记会影响到 l 和 r 。对 a 的加法则会对 l, r, s, t 均产生影响。只需要注意 a 不存在的边界情况即可（例如区间只有 a 的最大值与 a 存在）。

接下来需要考虑在 $pushup$ 时如何维护。我们可以考虑一下完成 a 最大值的更新之后，讨论左右儿子的 a 最大值是否与当前节点 a 最大值相等。我们以左儿子为例进行讲解，右儿子类似处理：

- 当左儿子的 a 最大值与当前节点的 a 最大值均相等时，左儿子的 a 会分别对当前节点的 a 产生贡献。

- 当左儿子的 最大值与当前节点 最大值相等，但是 最大值不相等时，左儿子的 会对该节点的 产生贡献， 会对该节点的 产生贡献。
- 当左儿子的 最大值与当前节点 最大值不相等，但是 最大值相等时，左儿子的 会对该节点的 产生贡献， 会对该节点的 产生贡献。
- 当左儿子的 最大值与当前节点的 最大值均不相等时，左儿子的 仅会对该节点的 产生贡献。

区间查询结果的 即为所求。

由于需要同时维护区间 和区间加法，所以复杂度仍是 。

```

1 #include <LTalgorithm>
2 #include <LTcstdint>
3 #include <LTiostream>
4 using i64 = int64_t;
5
6 constexpr int N = 300010;
7 constexpr i64 INF = 114514191981011451411;
8 int n, q, op, l, r;
9 int a[N], b[N];
10
11 int lc(int u) { return (u << 1); }
12
13 int rc(int u) { return (u << 1) | 1; }
14
15 struct node {
16     i64 mx_ab[2][2]; // 0: not max A or B; 1: is max A or B
17
18     i64 max_val() {
19         return std::max({mx_ab[0][0], mx_ab[0][1], mx_ab[1][0], mx_ab[1][1]});
20     }
21
22     i64 mx1[2], mx2[2]; // 0: A; 1: B
23     i64 tag_add[2], tag_mn[2];
24
25     void clr_tag() { tag_add[0] = tag_add[1] = 0, tag_mn[0] = tag_mn[1] = INF; }
26
27     void init(i64 a, i64 b) {
28         mx1[0] = a, mx1[1] = b, mx2[0] = mx2[1] = -INF;
29         mx_ab[1][1] = a + b, mx_ab[0][1] = mx_ab[1][0] = mx_ab[0][0] = -INF;
30     }
31
32     void modify_mn(i64 tg, bool id) // id 0: A; 1: B
33     {
34         if (mx1[id] <= tg) return;
35         mx_ab[1][1] += (mx_ab[1][1] > -INF) ? (tg - mx1[id]) : 0;
36         mx_ab[!id][id] += (mx_ab[!id][id] > -INF) ? (tg - mx1[id]) : 0;
37         mx1[id] = tag_mn[id] = tg;
38     }
39
40     void modify_add(i64 tg, bool id) // id 0: A; 1: B
41     {
42         for (int i = 0; i < 2; ++i)
43             for (int j = 0; j < 2; ++j) mx_ab[i][j] += (mx_ab[i][j] > -INF) ? tg : 0;
44         mx1[id] += tg, mx2[id] += (mx2[id] > -INF) ? tg : 0;
45         tag_mn[id] += (tag_mn[id] < INF) ? tg : 0, tag_add[id] += tg;
46     }
47
48     void pushup(const node& l, const node& r) {
49         for (int i = 0; i < 2; ++i) {
50             if (l.mx1[i] == r.mx1[i])
51                 mx1[i] = l.mx1[i], mx2[i] = std::max(l.mx2[i], r.mx2[i]);
52             else if (l.mx1[i] > r.mx1[i])

```

```

53     mx1[i] = l.mx1[i], mx2[i] = std::max(l.mx2[i], r.mx1[i]);
54     else
55         mx1[i] = r.mx1[i], mx2[i] = std::max(l.mx1[i], r.mx2[i]);
56 }
57 if (l.mx1[0] == mx1[0] && l.mx1[1] == mx1[1]) {
58     mx_ab[1][1] = l.mx_ab[1][1], mx_ab[0][1] = l.mx_ab[0][1];
59     mx_ab[1][0] = l.mx_ab[1][0], mx_ab[0][0] = l.mx_ab[0][0];
60 } else if (l.mx1[0] == mx1[0] && l.mx1[1] != mx1[1]) {
61     mx_ab[1][1] = mx_ab[0][1] = -INF;
62     mx_ab[1][0] = std::max(l.mx_ab[1][1], l.mx_ab[1][0]);
63     mx_ab[0][0] = std::max(l.mx_ab[0][1], l.mx_ab[0][0]);
64 } else if (l.mx1[0] != mx1[0] && l.mx1[1] == mx1[1]) {
65     mx_ab[1][1] = mx_ab[1][0] = -INF;
66     mx_ab[0][1] = std::max(l.mx_ab[1][1], l.mx_ab[0][1]);
67     mx_ab[0][0] = std::max(l.mx_ab[1][0], l.mx_ab[0][0]);
68 } else {
69     mx_ab[1][1] = mx_ab[0][1] = mx_ab[1][0] = -INF;
70     mx_ab[0][0] = std::max(
71         {l.mx_ab[0][0], l.mx_ab[0][1], l.mx_ab[1][0], l.mx_ab[1][1]});
72 }
73
74 if (r.mx1[0] == mx1[0] && r.mx1[1] == mx1[1]) {
75     mx_ab[1][1] = std::max(mx_ab[1][1], r.mx_ab[1][1]);
76     mx_ab[0][1] = std::max(mx_ab[0][1], r.mx_ab[0][1]);
77     mx_ab[1][0] = std::max(mx_ab[1][0], r.mx_ab[1][0]);
78     mx_ab[0][0] = std::max(mx_ab[0][0], r.mx_ab[0][0]);
79 } else if (r.mx1[0] == mx1[0] && r.mx1[1] != mx1[1]) {
80     mx_ab[1][0] = std::max({mx_ab[1][0], r.mx_ab[1][1], r.mx_ab[1][0]});
81     mx_ab[0][0] = std::max({mx_ab[0][0], r.mx_ab[0][1], r.mx_ab[0][0]});
82 } else if (r.mx1[0] != mx1[0] && r.mx1[1] == mx1[1]) {
83     mx_ab[0][1] = std::max({mx_ab[0][1], r.mx_ab[1][1], r.mx_ab[0][1]});
84     mx_ab[0][0] = std::max({mx_ab[0][0], r.mx_ab[1][0], r.mx_ab[0][0]});
85 } else
86     mx_ab[0][0] = std::max({mx_ab[0][0], r.mx_ab[0][0], r.mx_ab[0][1],
87         r.mx_ab[1][0], r.mx_ab[1][1]});
88 }
89 } tr[N << 2];
90
91 void pushup(int u) { tr[u].pushup(tr[lc(u)], tr[rc(u)]); }
92
93 void pushdown(int u) {
94     for (int i = 0; i < 2; ++i)
95         if (tr[u].tag_add[i])
96             tr[lc(u)].modify_add(tr[u].tag_add[i], i),
97             tr[rc(u)].modify_add(tr[u].tag_add[i], i);
98     for (int i = 0; i < 2; ++i)
99         if (tr[u].tag_mn[i] < INF)
100             tr[lc(u)].modify_mn(tr[u].tag_mn[i], i),
101             tr[rc(u)].modify_mn(tr[u].tag_mn[i], i);
102     tr[u].clr_tag();
103 }
104
105 void _build(int u, int L, int R) {
106     tr[u].clr_tag();
107     if (L == R) return tr[u].init(a[L], b[L]);
108     int M = (L + R) >> 1;
109     _build(lc(u), L, M), _build(rc(u), M + 1, R), pushup(u);
110 }
111
112 void build() { _build(1, 1, n); }
113
114 void _modify_mn(int u, int l, int r, int L, int R, int v, bool i) {
115     if (L > r || R < l || tr[u].mx1[i] <= v) return;
116     if (l <= L && R <= r && tr[u].mx2[i] < v) return tr[u].modify_mn(v, i);
117     pushdown(u);

```

```

118 int M = (L + R) >> 1;
119 _modify_mn(lc(u), l, r, L, M, v, i), _modify_mn(rc(u), l, r, M + 1, R, v, i);
120 pushup(u);
121 }
122
123 void modify_mn(int l, int r, int v, bool i) { _modify_mn(l, l, r, l, n, v, i); }
124
125 void _modify_add(int u, int l, int r, int L, int R, int v, bool i) {
126     if (L > r || R < l) return;
127     if (l <= L && R <= r) return tr[u].modify_add(v, i);
128     pushdown(u);
129     int M = (L + R) >> 1;
130     _modify_add(lc(u), l, r, L, M, v, i),
131     _modify_add(rc(u), l, r, M + 1, R, v, i);
132     pushup(u);
133 }
134
135 void modify_add(int l, int r, int v, bool i) {
136     _modify_add(l, l, r, l, n, v, i);
137 }
138
139 node _query(int u, int l, int r, int L, int R) {
140     if (l <= L && R <= r) return tr[u];
141     pushdown(u);
142     int M = (L + R) >> 1;
143     if (r <= M) return _query(lc(u), l, r, L, M);
144     if (l > M) return _query(rc(u), l, r, M + 1, R);
145     node ret;
146     node _l = _query(lc(u), l, M, L, M), _r = _query(rc(u), M + 1, r, M + 1, R);
147     ret.pushup(_l, _r);
148     return ret;
149 }
150
151 node query(int l, int r) { return _query(1, l, r, 1, n); }
152
153 using std::cin;
154 using std::cout;
155
156 int main() {
157     cin.tie(nullptr)->sync_with_stdio(false);
158     cin >> n >> q;
159     for (int i = 1; i <= n; ++i) cin >> a[i];
160     for (int i = 1; i <= n; ++i) cin >> b[i];
161     build();
162     while (q--) {
163         cin >> op >> l >> r;
164         int x;
165         switch (op) {
166             case 1:
167                 cin >> x;
168                 modify_mn(l, r, x, 0);
169                 break;
170             case 2:
171                 cin >> x;
172                 modify_mn(l, r, x, 1);
173                 break;
174             case 3:
175                 cin >> x;
176                 modify_add(l, r, x, 0);
177                 break;
178             case 4:
179                 cin >> x;
180                 modify_add(l, r, x, 1);
181                 break;
182             case 5:

```

```

183         cout << query(l, r).max_val() << '\n';
184         break;
185     default:
186         break;
187     }
188 }
189 }

```

小结

在第本章节中我们给出了四道例题，分别讲解了基本区间最值操作的维护、多个标记的优先级处理、数集分类的思想以及多个分类的维护。本质上处理区间最值的基本思想就是数集信息的分类维护与高效合并。在下一章节中，我们将探讨历史区间最值的相关问题。

历史最值问题

历史最值不等于可持久化

注意，本章所讲到的历史最值问题不同于所谓的可持久化数据结构。这类特殊的问题我们将其称为历史最值问题。历史最值的问题可以分为三类。

历史最大值

简单地说，一个位置的历史最大值就是当前位置下曾经出现过的数的最大值。形式化地定义，我们定义一个辅助数组，一开始与 完全相同。在 的每次操作后，我们对整个数组取：

这时，我们将 称作这个位置的历史最大值，

历史最小值

定义与历史最大值类似，在 的每次操作后，我们对整个数组取 。这时，我们将 称作这个位置的历史最小值，

历史版本和

辅助数组 一开始全部是 。在每一次操作后，我们把整个 数组累加到 数组上

我们称 为 这个位置上的历史版本和。

接下来，我们将历史最值问题分成四类讨论。

可以用标记处理的问题

▼ CPU 监控

序列 一开始相同：

1. 对 做区间覆盖
2. 对 做区间加
3. 询问 的区间
4. 询问 的区间

每次操作后，我们都进行一次更新，。。

我们先不考虑操作 1。那么只有区间加的操作，我们维护标记 表示当前区间增加的值，这个标记可以解决区间 的问题。接下来考虑历史区间。我们定义标记，该标记的含义是：在该标记的生存周期内，标记的历史最大值。

这个定义可能比较模糊。因此我们先解释一下标记的生存周期。一个标记会经历这样的过程：

1. 在结点 被建立。
2. 在结点 接受若干个新的标记的同时，与新的标记合并（指同类标记）
3. 结点 的标记下传给 的儿子， 的标记清空

我们认为在这个过程中，从 1 开始到 3 之前，都是结点 的标记的生存周期。两个标记合并后，成为同一个标记，那么他们的生存周期也会合并（即取建立时间较早的那个做为生存周期的开始）。一个与之等价的说法是，从上次把这个结点的标记下传的時刻到当前時刻这一时间段。

为什么要定义生存周期？利用这个概念，我们可以证明：在一个结点标记的生存周期内，其子结点均不会发生任何变化，并保留在这个生存周期之前的状态。道理很简单，因为在这个期间你是没有下传标记的。

于是，你就可以保证，在当前标记生存周期内的历史 的最大值是可以更新到子结点的标记和信息上的。因为子结点的标记和信息在这个时间段内都没有变过。于是我们把 的标记下传给它的儿子，不难发现

那么信息的更新也是类似的，拿对应的标记更新即可。

接下来，我们考虑操作 1。

区间覆盖操作，会把所有的数变成一个数。在这之后，无论是区间加减还是覆盖，整个区间的数仍是同一个（除非你结束当前标记的生存周期，下传标记）。因此我们可以把第一次区间覆盖后的所有标记都看成区间覆盖标记。也就是说一个标记的生存周期被大致分成两个阶段：

1. 若干个加减操作标记的合并，没有接收过覆盖标记。
2. 覆盖操作的标记，没有所谓的加减标记（加减标记转化为覆盖标记）

于是我们把这个结点的 Pre 标记拆成。表示第一阶段的最大加减标记；表示第二阶段的最大覆盖标记。利用相似的方法，我们可以对这个做标记下传和信息更新。时间复杂度是 的（这个问题并没有区间对 取最值的操作哦～）

```
1 #include <LTAgorithm>
2 #include <LTclimits>
3 #include <LTiostream>
4 using namespace std;
5 using ll = long long;
6
7 constexpr int N = 1e5 + 7;
8
9 struct Tree {
10     int mx, _mx; // 区间最大值 区间历史最大值
11     int ad, _ad; // 区间加标记 区间阶段历史最大加标记
12     int st, _st; // 区间修改值 区间阶段历史最大修改标记
13 } g[N * 4];
14
15 int a[N];
16 int n, m;
17 #define ls u * 2
18 #define rs u * 2 + 1
19 #define mid (l + r) / 2
20
21 void pushup(int u) {
22     g[u].mx = max(g[ls].mx, g[rs].mx);
23     g[u]._mx = max(g[ls]._mx, g[rs]._mx);
24 }
25
26 void pushadd(int u, int v, int _v) {
```

```

27  g[u]._mx = max(g[u]._mx, g[u].mx + _v), g[u].mx += v;
28  if (g[u].st == INT_MIN)
29      g[u]._ad = max(g[u]._ad, g[u].ad + _v), g[u].ad += v;
30  else
31      g[u]._st = max(g[u]._st, g[u].st + _v), g[u].st += v;
32 }
33
34 void pushset(int u, int v, int _v) {
35     g[u]._mx = max(g[u]._mx, _v), g[u].mx = v;
36     g[u]._st = max(g[u]._st, _v), g[u].st = v;
37 }
38
39 void pushdown(int u, int l, int r) {
40     if (g[u].ad || g[u]._ad)
41         pushadd(ls, g[u].ad, g[u]._ad), pushadd(rs, g[u].ad, g[u]._ad),
42         g[u].ad = g[u]._ad = 0;
43     if (g[u].st != INT_MIN || g[u]._st != INT_MIN)
44         pushset(ls, g[u].st, g[u]._st), pushset(rs, g[u].st, g[u]._st),
45         g[u].st = g[u]._st = INT_MIN;
46 }
47
48 void build(int u = 1, int l = 1, int r = n) {
49     g[u]._st = g[u].st = INT_MIN;
50     if (l == r) {
51         g[u].mx = a[l];
52         g[u]._mx = a[l];
53         return;
54     }
55     build(ls, l, mid), build(rs, mid + 1, r);
56     pushup(u);
57 }
58
59 int L, R;
60
61 void add(int v, int u = 1, int l = 1, int r = n) {
62     if (R < l || r < L) return;
63     if (L <= l && r <= R) return pushadd(u, v, max(v, 0));
64     pushdown(u, l, r);
65     add(v, ls, l, mid), add(v, rs, mid + 1, r);
66     pushup(u);
67 }
68
69 void tset(int v, int u = 1, int l = 1, int r = n) {
70     if (R < l || r < L) return;
71     if (L <= l && r <= R) return pushset(u, v, v);
72     pushdown(u, l, r);
73     tset(v, ls, l, mid), tset(v, rs, mid + 1, r);
74     pushup(u);
75 }
76
77 int qmax(int u = 1, int l = 1, int r = n) {
78     if (R < l || r < L) return INT_MIN;
79     if (L <= l && r <= R) return g[u].mx;
80     pushdown(u, l, r);
81     return max(qmax(ls, l, mid), qmax(rs, mid + 1, r));
82 }
83
84 int qmaxh(int u = 1, int l = 1, int r = n) {
85     if (R < l || r < L) return INT_MIN;
86     if (L <= l && r <= R) return g[u]._mx;
87     pushdown(u, l, r);
88     return max(qmaxh(ls, l, mid), qmaxh(rs, mid + 1, r));
89 }
90
91 int main() {

```

```
92  cin.tie(nullptr)->sync_with_stdio(false);
93  cin >> n;
94  for (int i = 1; i <= n; ++i) cin >> a[i];
95  build();
96  int m, z;
97  cin >> m;
98  for (int i = 1; i <= m; ++i) {
99      char op;
100     cin >> op;
101     while (op == ' ' || op == '\r' || op == '\n') cin >> op;
102     cin >> L >> R;
103     int x;
104     if (op == 'Q')
105         cout << qmax() << '\n';
106     else if (op == 'A')
107         cout << qmaxh() << '\n';
108     else if (op == 'P')
109         cin >> x, add(x);
110     else
111         cin >> x, tset(x);
112 }
113 return 0;
114 }
```

本页面最近更新：2025/4/6 19:36:35, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [CCXXI](#), [countercurrent-time](#), [Enter-tainer](#), [GoatGirl98](#), [H-J-Granger](#), [hsfzLZH1](#), [ImpleLee](#), [kenlig](#), [lanruixiang](#), [NachtgeistW](#), [opsiff](#), [ouuan](#), [sshwy](#), [StudyingFather](#), [SukkaW](#), [Tiphereth-A](#), [Xeonacid](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用